Лекция 3

Базовые конструкции языка Python

Дисциплина «Информационные технологии и программирование»

План лекции

1. **Модель выполнения Python-программы** — интерпретатор, скрипты, запуск
2. **Синтаксис и структура кода** — отступы, инструкции, комментарии
3. **Переменные и основные типы данных** — `int`, `float`, `str`, `bool`
4. **Операции и выражения** — арифметика, приоритет, преобразование типов
5. **Ввод и вывод** — `input()`, `print()`, f-строки
6. **Простейшие вычислительные алгоритмы** — пошаговый практический пример

1. Модель выполнения Python-программы

- Python — **интерпретируемый** язык: код выполняется построчно интерпретатором
- Предварительная компиляция в машинный код не требуется
- Исходный код — сценарий (скрипт) в файле с расширением `.py`
- Запуск из терминала:

```
python main.py
# или
python3 main.py
```

- Проверка установки: `python --version`

2. Синтаксис и структура кода

- Инструкции выполняются последовательно, сверху вниз
- Блоки кода выделяются **отступами** (PEP 8 рекомендует 4 пробела)
- Комментарии начинаются с `#` и игнорируются интерпретатором
- Читаемость кода — один из принципов языка

```
# Комментарий: интерпретатор эту строку не выполняет
x = 2 + 2   # комментарий может стоять и после кода
```

3. Переменные и основные типы данных

Переменная — именованная область памяти для хранения значения.

Типизация **динамическая**: тип определяется по присвоенному значению.

```
distance = 250.5    # float — число с дробной частью
count    = 10       # int   — целое число
name     = "Маршрут"  # str   — строка (текст в кавычках)
flag     = True     # bool  — логическое значение
```

Правила именования переменных

- Допустимы буквы, цифры и `_`; имя **не может начинаться с цифры**
- Регистр значим: `Price` и `price` — разные переменные
- Пробелы и спецсимволы (`-`, `@`, `#`) запрещены
- Стиль PEP 8 — snake_case: `fuel_cost`, `liters_per_100`

✅ Корректно: `distance`, `liters_per_100`, `price2`
❌ Некорректно: `2price`, `my var`, `cost-price`

4. Ввод и вывод данных

```
print("Расчёт начат")                    # вывод в консоль
distance = input("Введите расстояние: ")  # ввод от пользователя
```

- `print()` выводит значения в консоль через пробел
- `input()` показывает приглашение и ждёт ввода пользователя
- ⚠️ **`input()` всегда возвращает строку (`str`)** — даже если введено число

Преобразование типов

```
s = input("Введите число: ")   # s == "250.5" — это строка
x = float(s)                   # x == 250.5  — теперь число
n = int("100")                 # n == 100    — целое число
```

- `float()` — преобразование строки в дробное число
- `int()` — преобразование строки в целое число
- Без преобразования арифметика над введёнными данными невозможна

5. Арифметические операции

Оператор	Назначение	Пример
`+ - * /`	Сложение, вычитание, умножение, деление	`7 / 2 → 3.5`
`//`	Целочисленное деление	`7 // 2 → 3`
`%`	Остаток от деления	`7 % 2 → 1`
`**`	Возведение в степень	`2 ** 3 → 8`

Приоритет: скобки → `**` → `* / // %` → `+ -`

6. Форматированный вывод: f-строки

```
liters = 18.537
print(f"Потребуется: {liters:.2f} л.")
# Вывод: Потребуется: 18.54 л.
```

- Префикс `f` перед кавычками разрешает подстановку значений в `{}`
- Спецификатор `:.2f` — вывод с **двумя знаками после запятой** с округлением

Практический пример: постановка задачи

Задача. Рассчитать количество топлива на поездку и его стоимость.

Входные данные: расстояние (км), расход (л/100 км), цена литра (руб.).

Формулы:

```
liters = distance * consumption / 100
cost   = liters * price
```

Выход: два числа, выведенные с двумя знаками после запятой.

Шаг 1. Файл и комментарии

Создаём файл `main.py` и начинаем с комментария-«шапки»:

```
# Программа рассчитывает количество топлива на поездку
# и стоимость этого топлива.
# Входные данные:
# distance - расстояние поездки в километрах;
# consumption - расход топлива в литрах на 100 км;
# price - цена одного литра топлива.
```

Шаг 2. Запрос данных у пользователя

```
distance = float(input("Введите расстояние поездки в км: "))
consumption = float(input("Введите расход топлива в литрах на 100 км: "))
price = float(input("Введите цену одного литра топлива: "))
```

- `input()` возвращает строку → сразу преобразуем её через `float()`
- Выбран `float`, а не `int`: расстояние, расход и цена могут быть дробными

Шаг 3. Вычисления

```
# чтобы найти расход на поездку, расстояние умножаем
# на норму расхода и делим на 100
liters = distance * consumption / 100

# количество литров умножаем на цену одного литра
cost = liters * price
```

- Расчёт разбит на два шага для читаемости
- Промежуточные результаты хранятся в переменных `liters` и `cost`

Шаг 4. Вывод результатов

```
print(f"На поездку потребуется {liters:.2f} л топлива.")
print(f"Стоимость топлива составит {cost:.2f} руб.")
```

Результат работы программы:

```
На поездку потребуется 18.54 л топлива.
Стоимость топлива составит 1091.83 руб.
```

Полный код программы

```
# Программа рассчитывает количество топлива на поездку и его стоимость.
distance = float(input("Введите расстояние поездки в км: "))
consumption = float(input("Введите расход топлива в литрах на 100 км: "))
price = float(input("Введите цену одного литра топлива: "))

liters = distance * consumption / 100
cost = liters * price

print(f"На поездку потребуется {liters:.2f} л топлива.")
print(f"Стоимость топлива составит {cost:.2f} руб.")
```

Тестирование программы

№	Ввод: расстояние, расход, цена	Ожидаемый вывод
1	100, 8, 60	8.00 л и 480.00 руб.
2	250.5, 7.4, 58.90	18.54 л и 1091.83 руб.
3	0, 10, 50	0.00 л и 0.00 руб.

- Проверяем целые значения, дробные значения и граничный случай (ноль)
- Набор 2 вручную: `250.5 * 7.4 / 100 = 18.537 → 18.54`

Типичные ошибки

- Забыто преобразование: `input()` без `float()` → ошибка при арифметике
- Забыт префикс `f`: `"{liters:.2f}"` выводится как обычный текст
- Перепутаны `/` и `//` → неверный целочисленный результат
- Нарушены отступы → `IndentationError`
- Перепутан порядок запроса входных данных

Итоги лекции

- Python-программа — сценарий, выполняемый интерпретатором построчно
- Переменные с динамической типизацией: `int`, `float`, `str`, `bool`
- `input()` возвращает строку → данные преобразуем через `int()` / `float()`
- Схема последовательного алгоритма: **ввод** → **вычисление** → **вывод**
- f-строки и `:.2f` — удобный форматированный вывод результатов

Вопросы для самопроверки

1. Почему `input()` почти всегда оборачивают в `float()` или `int()`?
2. Чем отличаются операции `/`, `//` и `%`?
3. Что означает запись `{cost:.2f}` внутри f-строки?
4. Какие правила именования переменных действуют в Python?
5. Как запустить скрипт из командной строки и проверить установку Python?